

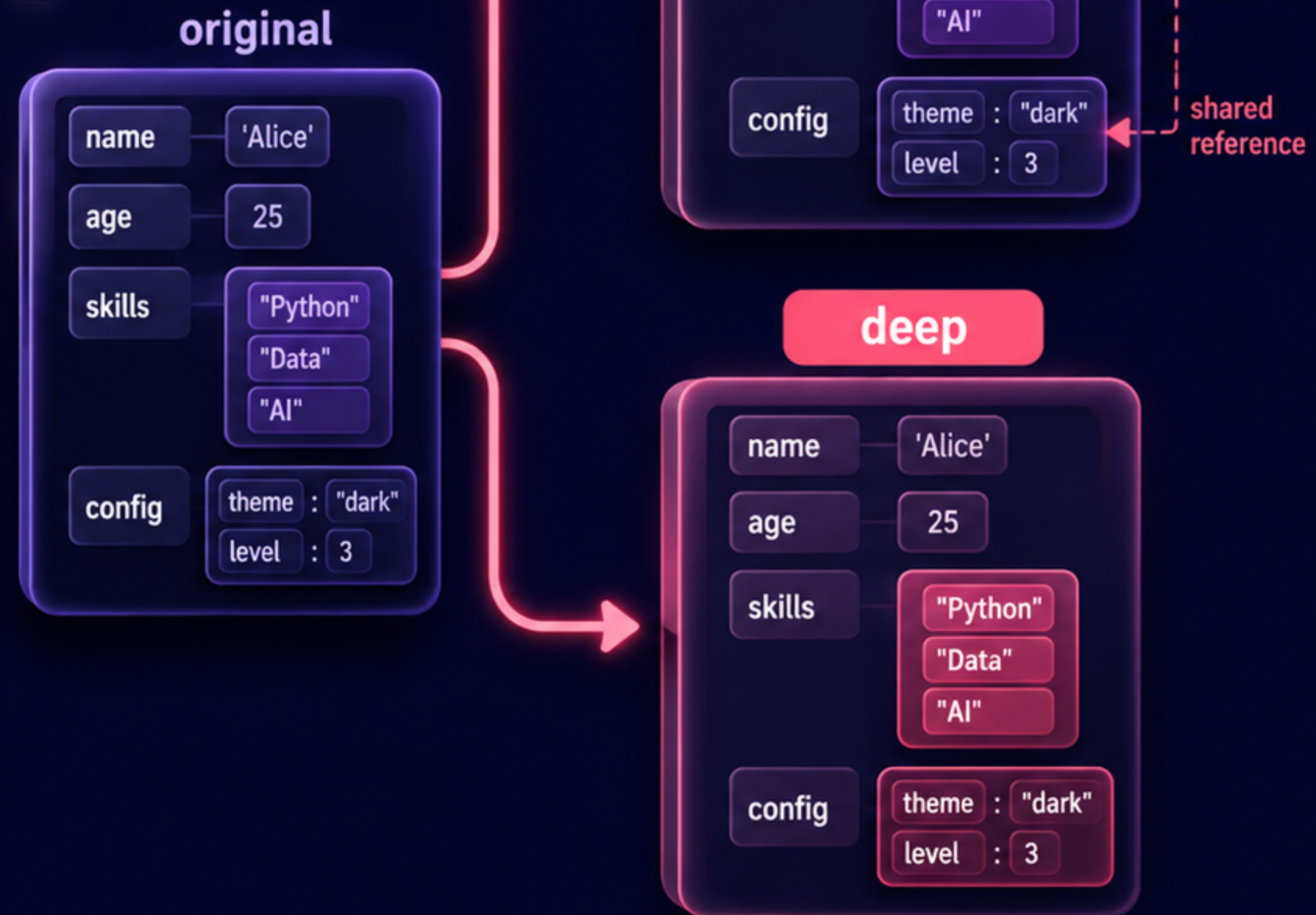
SHALLOW COPY VS DEEP COPY

Duplicate objects without surprises



Code and Debug

Python for Beginners



The = Assignment Trap



```
original = [1, 2, 3]
copy = original    # NOT a copy!

copy.append(4)
print(original)    # [1, 2, 3, 4]  <- surprise!
print(copy)        # [1, 2, 3, 4]
```

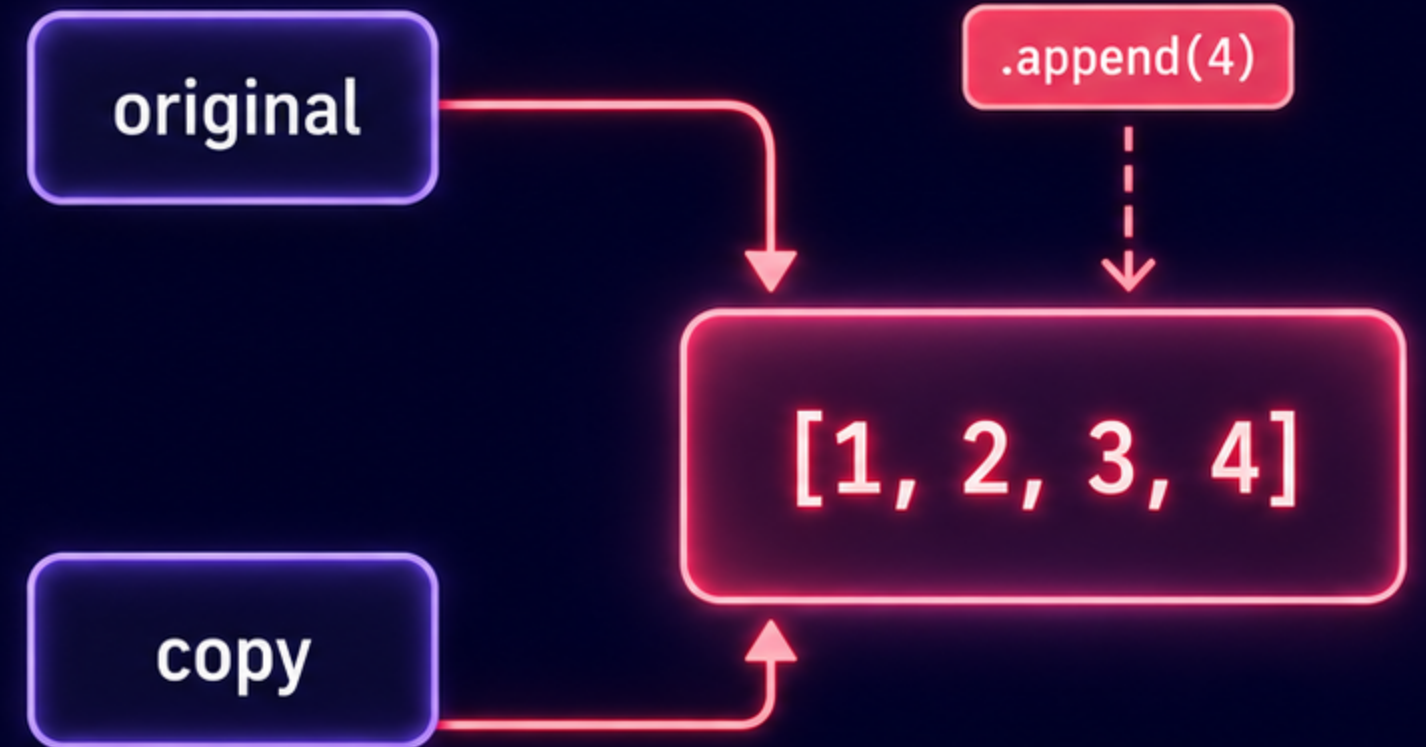
What = actually does:

- Creates a new name that points to the same object
- Both names now refer to one list in memory
- Modifying through one name affects the other



Quick Tip:

Assignment in Python is aliasing, not copying.
To actually copy, you need explicit copy methods.



ONE OBJECT, TWO NAMES.

Detecting Aliases with `id()` and `is`

Two tools to check whether two names point to the same object:



```
original = [1, 2, 3]
copy = original

print(id(original))      # 140234567890
print(id(copy))          # 140234567890
print(original is copy)  # True
```

- `id(obj)` - returns the memory address of an object
- `is` - checks if two names point to the SAME object (not just equal values)

Compare with `==`:



```
a = [1, 2, 3]
b = [1, 2, 3]

print(a == b)      # True
print(a is b)      # False
```

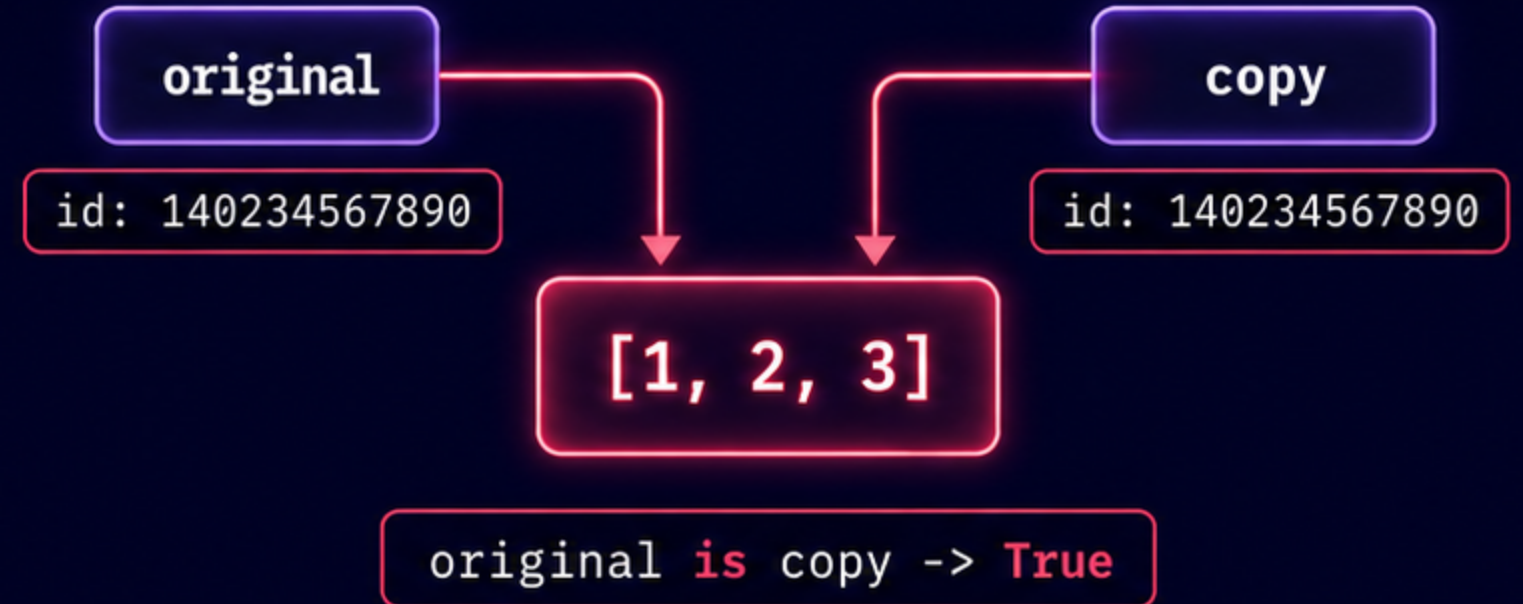
- `==` checks if the values are equal
- `is` checks if it's the same object



Quick Tip: Use `is` for identity, `==` for value equality.

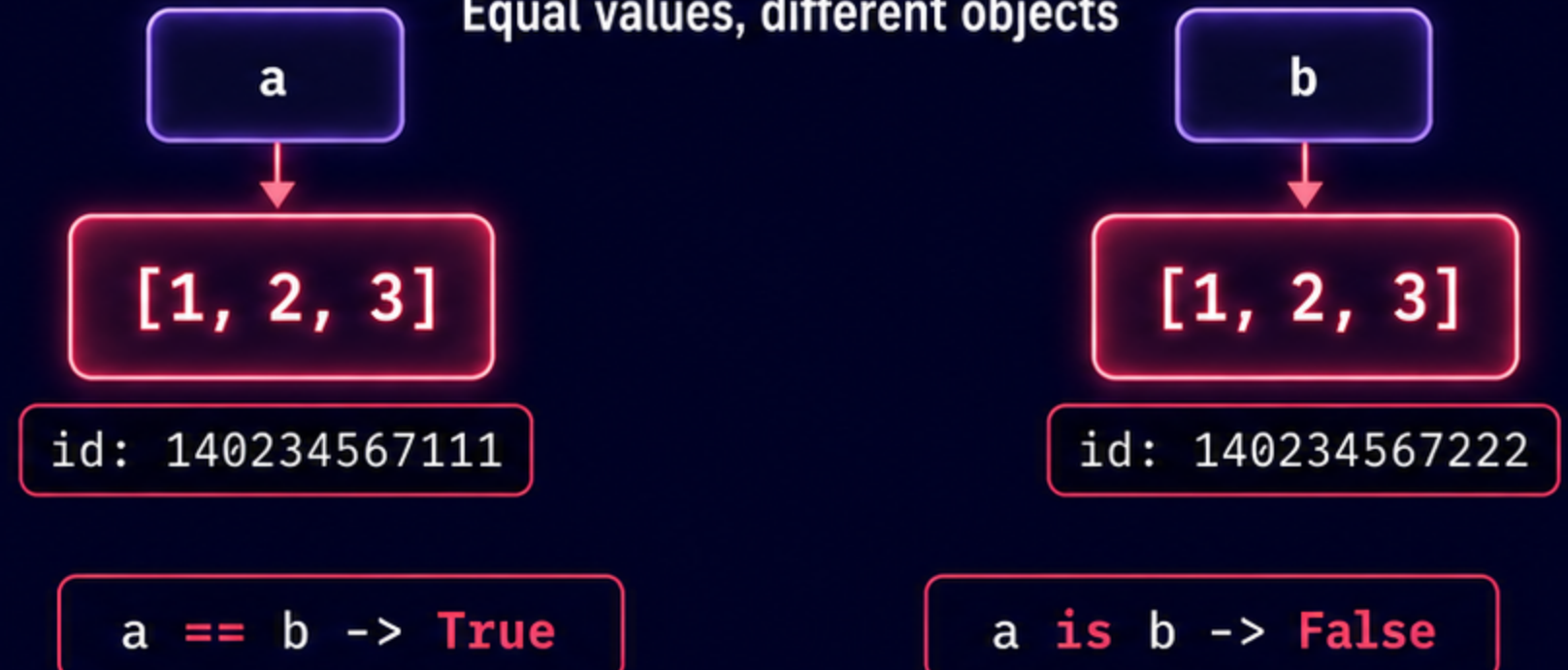
Scenario A

Aliases



Scenario B

Equal values, different objects



Shallow Copy with `copy.copy()`

```
import copy

original = [1, 2, 3]
shallow = copy.copy(original)

print(original is shallow)  # False - different objects
print(original == shallow)  # True - same contents

shallow.append(4)
print(original)             # [1, 2, 3]   # unchanged
print(shallow)              # [1, 2, 3, 4]
```

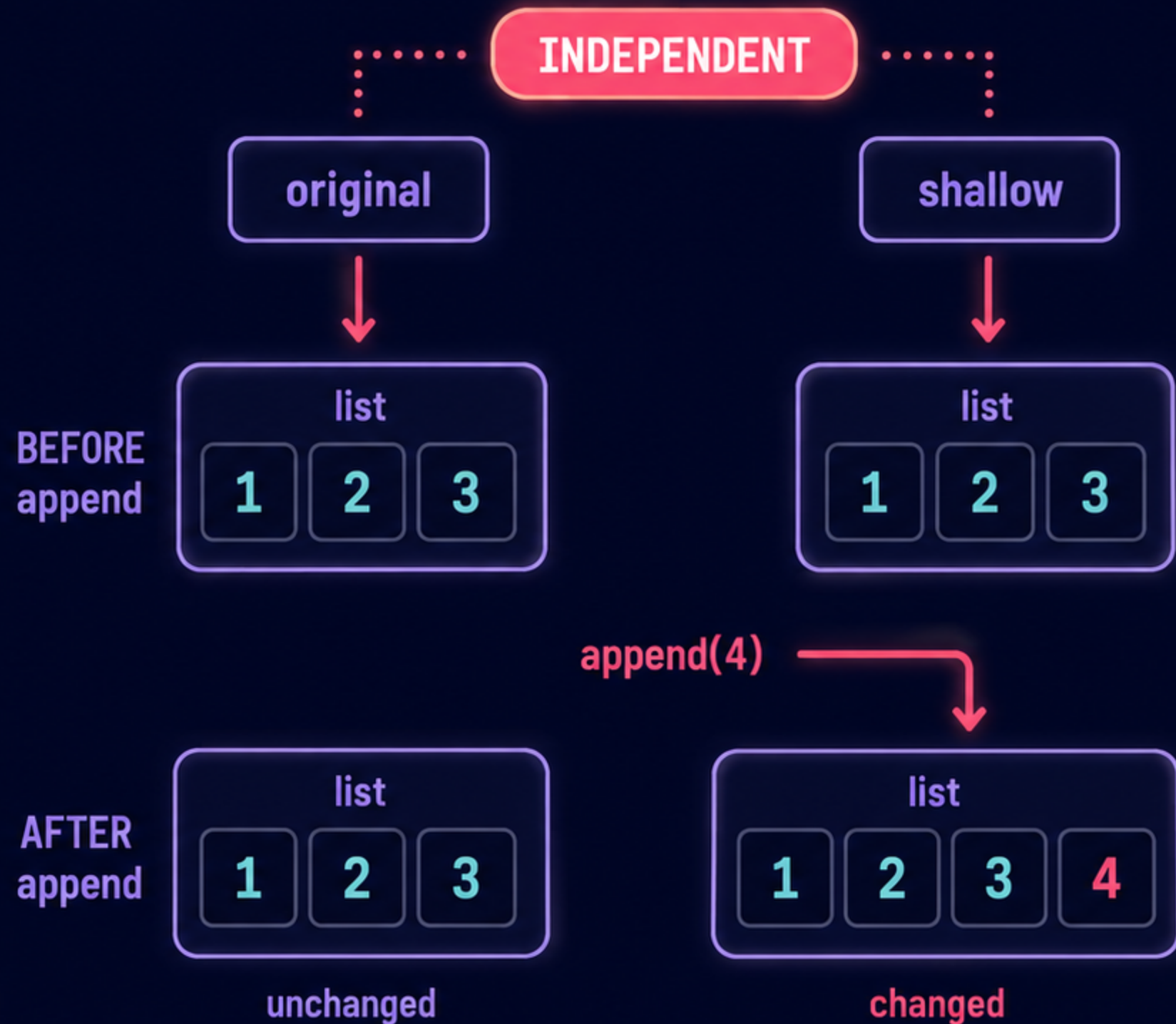
Other ways to make a shallow copy:

```
list_copy = original.copy()      # .copy() method
list_copy = original[:]          # slice trick
list_copy = list(original)       # constructor
```

For dictionaries: `my_dict.copy()` or `copy.copy(my_dict)`.



Note: For flat (non-nested) data, shallow copy is all you need.



The Shallow Copy Gotcha

Shallow copy only copies the outer container. Nested objects are shared.



```
import copy
```

```
original = [[1, 2], [3, 4]]
shallow = copy.copy(original)
```

```
# Modifying the OUTER list - independent
shallow.append([5, 6])
```

```
print(original)    # [[1, 2], [3, 4]]          unchanged
print(shallow)     # [[1, 2], [3, 4], [5, 6]]
```

```
# Modifying a NESTED list - affects both!
```

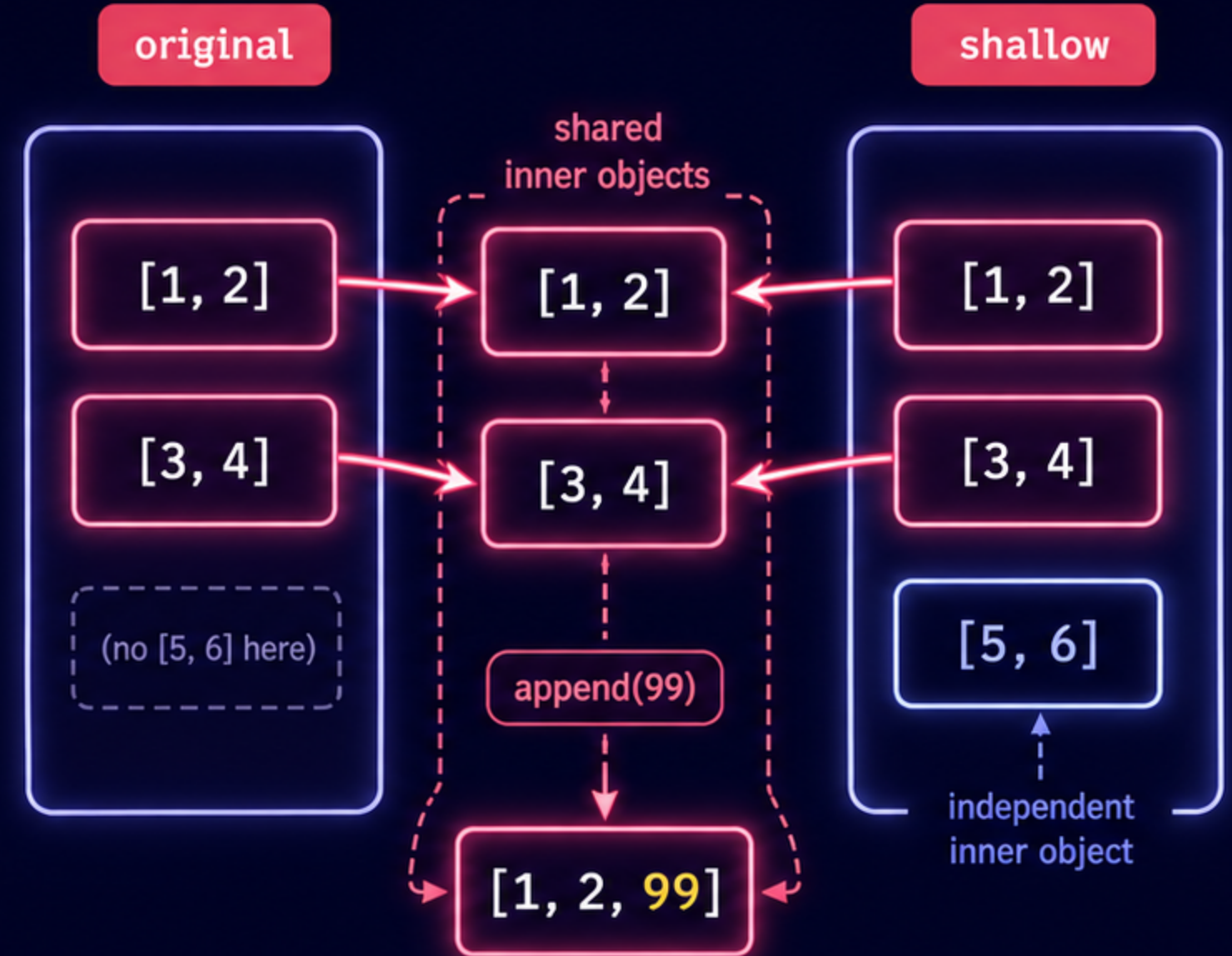
```
shallow[0].append(99)
print(original)    # [[1, 2, 99], [3, 4]]      surprise!
print(shallow)     # [[1, 2, 99], [3, 4], [5, 6]]
```

Why this happens:

- 1. The outer list is a new list
- 2. The inner lists are shared references
- 3. Mutating an inner list affects both outer lists



Watch Out: Shallow copy can be dangerous for nested data structures.



NEW OUTER LISTS – SHARED INNER OBJECTS

Deep Copy with `copy.deepcopy()`

A deep copy recursively copies everything - outer AND all nested objects.



```
import copy

original = [[1, 2], [3, 4]]
deep = copy.deepcopy(original)

# Modifying any level is now safe
deep[0].append(99)
print(original)      # [[1, 2], [3, 4]]
print(deep)          # [[1, 2, 99], [3, 4]]
```

How deep copy works:

- Recursively copies every level of the structure
- Inner lists, dicts, sets, custom objects - ALL get fresh copies
- Result: two completely independent structures



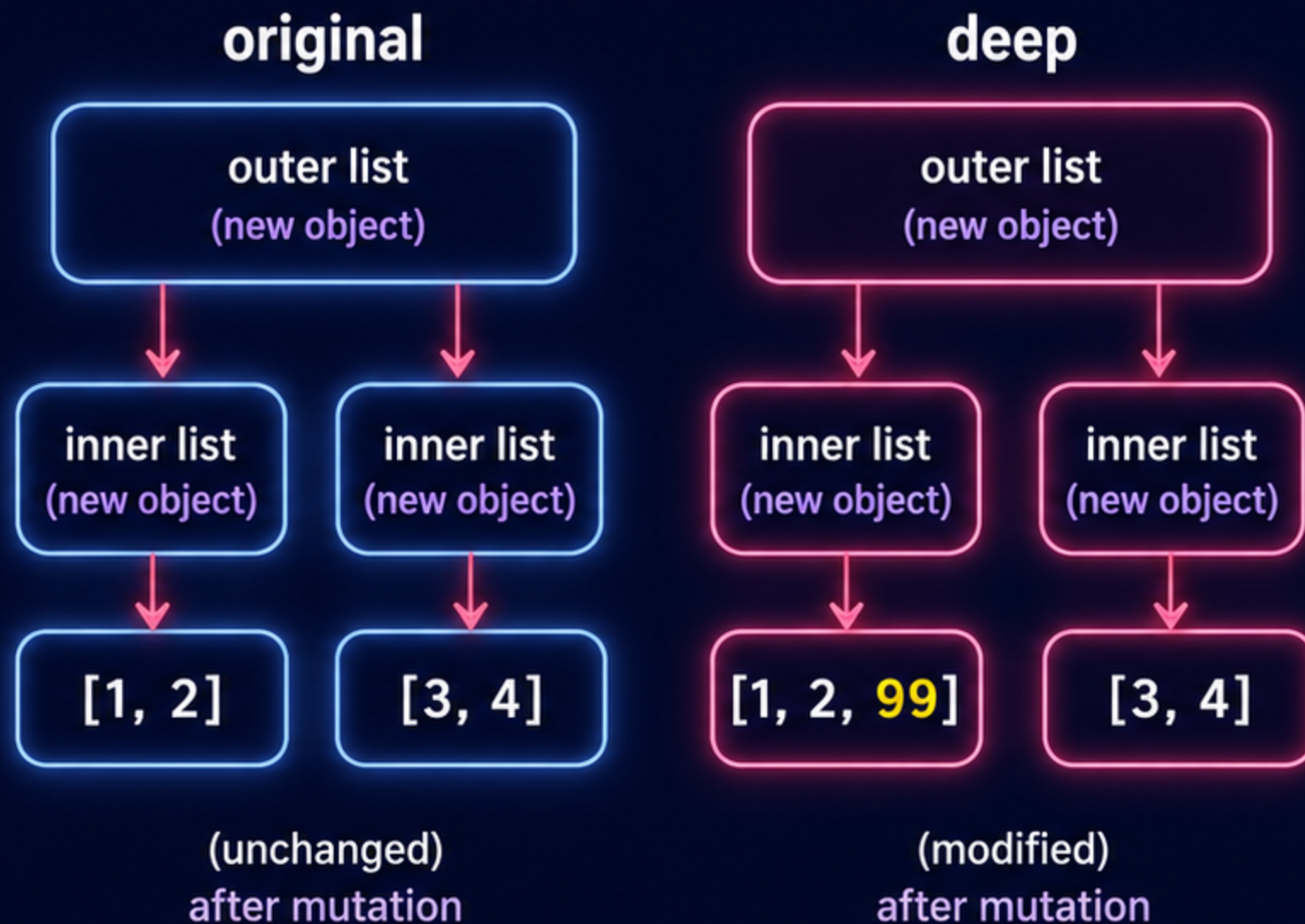
```
original = {"users": [{"name": "Anirudh"}]}
deep = copy.deepcopy(original)

deep["users"][0]["name"] = "Changed"
print(original)      # {'users': [{'name': 'Anirudh'}]}
```



Safe for nested data - the original stays untouched.

100% INDEPENDENT



✧ ----- NEW OUTER OBJECTS + NEW INNER OBJECTS ----- ✧

Shallow vs Deep: Decision Guide

Scenario	Use
Flat list/dict (no nesting)	Shallow copy (faster)
Nested data you'll modify	Deep copy (safe)
Only modifying the outer level	Shallow copy
Modifying ANY nested level	Deep copy
Performance critical + no nesting	Shallow copy

 **Quick decision rule:**
"Will I touch anything nested? Yes -> deep copy. No -> shallow copy."

-  **Cost:**
- Shallow copy is fast (one level of copying)
 - Deep copy is slower (recursive - proportional to structure size)



Quick Tip: When in doubt, default to deep copy. Optimize to shallow only if performance actually matters.

Best Practices + Common Pitfalls

DO

- ✓ Always use explicit copy methods - = is never a copy
- ✓ Use `copy.deepcopy()` when in doubt
- ✓ Use shallow copy for flat structures (faster)
- ✓ Use `is` to verify if you have a true copy vs an alias
- ✓ Test your copies - modify and check

DON'T

- ✗ Use = and expect a copy
- ✗ Use shallow copy on nested data you plan to mutate
- ✗ Deep-copy huge objects unnecessarily (it's slow)
- ✗ Forget that dicts, sets, and custom objects also need explicit copying



Quick Tip:

Bugs from accidental aliasing are some of the hardest to debug - they often appear far away from the original = line. Build the copy habit early.